

routine load manual

routine load manual

Routine Load 是 Apache Doris 提供的一种**流式导入**作业，用于从 Kafka Topic 中持续消费数据并写入 Doris 表。提交 Routine Load 作业后，Doris 会持续运行该导入作业，实时生成导入任务消费 Kafka 集群中指定 Topic 的消息，并提供 **Exactly-Once** 语义，保证数据不丢不重。

快速导航

针对不同诉求，可直接跳转到对应章节：

- 我的诉求：立刻跑通一个最小示例；跳转章节：快速上手
- 我的诉求：了解原理、状态机与自动恢复机制；跳转章节：基本原理
- 我的诉求：查询所有可配置参数（FE/BE/作业/Kafka）；跳转章节：参考手册
- 我的诉求：按场景查找示例（容错率、过滤、JSON、Kerberos 等）；跳转章节：导入示例
- 我的诉求：作业进入 PAUSED、报 out of range、SSL 错误；跳转章节：常见问题（FAQ）

使用场景

Routine Load 适用于以下场景：

- 需要将 Kafka Topic 中的实时数据持续同步到 Doris；
- 需要 **Exactly-Once** 语义保证，避免数据丢失或重复；
- 需要对导入数据进行列映射、过滤、衍生列计算等转换处理；
- 需要将一个 Kafka Topic 的数据动态分发到多个 Doris 表（一流多表）。

支持的数据源与格式

Routine Load 仅支持从 **Kafka 集群**消费数据，并支持以下两种消息格式：

- 格式：CSV；说明：每条 message 为一行，行尾**不包含**换行符。
- 格式：JSON；说明：单条 JSON 对象或包含多对象的 JSON 数组。

在导入 CSV 格式时，需要明确区分空值（null）与空字符串（''）：

- 空值（null）需要用 \n 表示，例如 a,\n,b 表示中间列是一个空值（null）；
- 空字符串（''）直接将数据置空，例如 a,,b 表示中间列是一个空字符串（''）。

使用限制

在使用 Routine Load 消费 Kafka 中数据时，有以下限制：

- 限制项：消息格式；说明：仅支持 CSV 及 JSON 文本格式。CSV 每一条 message 为一行，且行尾**不包含**换行符。
- 限制项：Kafka 版本；说明：默认支持 Kafka 0.10.0.0（含）以上版本。如果要使用 Kafka 0.10.0.0 以下版本（0.9.0、0.8.2、0.8.1、0.8.0），需要修改 BE 的配置 kafka_broker_version_fallback 为要兼容的旧版本，或者在创建作业时直接设置 property.broker.version.fallback。使用旧版本的代价是 Routine Load 的部分新特性可能无法使用，如根据时间设置 Kafka 分区的 offset。

基本原理

Routine Load 会持续消费 Kafka Topic 中的数据，写入 Doris 中。在 Doris 中，创建 Routine Load 作业后会生成一个常驻的导入作业，包括若干个导入任务：

- **导入作业（Load Job）**：一个 Routine Load Job 是一个常驻的导入作业，会持续不断地消费数据源中的数据。
- **导入任务（Load Task）**：一个导入作业会被拆解成若干个导入任务进行实际消费，每个任务都是一个独立的事务。

Routine Load 的导入具体流程如下图所示：

图片说明：Routine Load

整体流程如下：

1. Client 向 FE 提交创建 Routine Load 作业请求，FE 通过 Routine Load Manager 生成一个常驻的导入作业（Routine Load Job）。
2. FE 通过 Job Scheduler 将 Routine Load Job 拆分成若干个 Routine Load Task，由 Task Scheduler 进行调度，下发到 BE 节点。
3. 在 BE 上，一个 Routine Load Task 导入完成后向 FE 提交事务，并更新 Job 的元数据。
4. 一个 Routine Load Task 提交后，会继续生成新的 Task，或对超时的 Task 进行重试。
5. 新生成的 Routine Load Task 由 Task Scheduler 继续调度，不断循环。

自动恢复机制

为了确保作业的高可用性，Routine Load 引入了自动恢复机制。在非预期暂停的情况下，Routine Load Scheduler 调度线程会尝试自动恢复作业。对于 Kafka 侧的意外宕机或其他无法工作的情况，自动恢复机制可以确保在 Kafka 恢复后，无需人工干预，导入作业能够继续正常运行。

下表列出了哪些情况会自动恢复，哪些需要人工介入：

- 触发暂停的原因：Kafka broker 临时不可达、网络抖动；是否自动恢复：是；处理建议：Scheduler 会按 `period_of_auto_resume_min` 周期自动重试
- 触发暂停的原因：用户手动执行 `PAUSE ROUTINE LOAD`；是否自动恢复：否；处理建议：需要手动 `RESUME ROUTINE LOAD`
- 触发暂停的原因：数据质量问题（超过 `max_filter_ratio` 或 `max_error_number`）；是否自动恢复：否；处理建议：排查 `ErrorLogUrls`，调整数据或参数后 `RESUME`
- 触发暂停的原因：库表被删除等无法恢复的元数据异常；是否自动恢复：否；处理建议：重新创建表或重建作业

快速上手

本节通过一个最小可运行示例，演示从 Kafka 创建 Routine Load 作业，并完成作业的查看、暂停、恢复、修改、停止等基本操作。

创建导入作业

可以通过 `CREATE ROUTINE LOAD` 命令创建常驻 Routine Load 导入任务。Routine Load 可以消费 CSV 和 JSON 数据。

导入 CSV 数据

1. 准备 Kafka 数据样本：

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test-routine-load-csv --from-beginning
1,Emily,25
2,Benjamin,35
3,Olivia,28
4,Alexander,60
5,Ava,17
6,William,69
7,Sophia,32
8,James,64
```

9,Emma,37
10,Liam,64

2. 创建导入目标表：

```
CREATE TABLE testdb.test_routineload_tbl(  
  user_id    BIGINT    NOT NULL COMMENT "user id",  
  name       VARCHAR(20)    COMMENT "name",  
  age        INT       COMMENT "age"  
)  
DUPLICATE KEY(user_id)  
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

3. 创建 Routine Load 导入作业：

```
CREATE ROUTINE LOAD testdb.example_routine_load_csv ON test_routineload_tbl  
COLUMNS TERMINATED BY ",",  
COLUMNS(user_id, name, age)  
FROM KAFKA(  
  "kafka_broker_list" = "192.168.88.62:9092",  
  "kafka_topic" = "test-routine-load-csv",  
  "property.kafka_default_offsets" = "OFFSET_BEGINNING"  
);
```

导入 JSON 数据

1. 准备 Kafka 数据样本：

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test-routine-  
load-json --from-beginning  
{ "user_id":1,"name":"Emily","age":25}  
{ "user_id":2,"name":"Benjamin","age":35}  
{ "user_id":3,"name":"Olivia","age":28}  
{ "user_id":4,"name":"Alexander","age":60}  
{ "user_id":5,"name":"Ava","age":17}  
{ "user_id":6,"name":"William","age":69}  
{ "user_id":7,"name":"Sophia","age":32}  
{ "user_id":8,"name":"James","age":64}  
{ "user_id":9,"name":"Emma","age":37}  
{ "user_id":10,"name":"Liam","age":64}
```

2. 创建导入目标表：

```
CREATE TABLE testdb.test_routineload_tbl(
  user_id      BIGINT  NOT NULL COMMENT "user id",
  name         VARCHAR(20)  COMMENT "name",
  age          INT       COMMENT "age"
)
DUPLICATE KEY(user_id)
DISTRIBUTED BY HASH(user_id) BUCKETS 10;
```

3. 创建 Routine Load 导入作业：

```
CREATE ROUTINE LOAD testdb.example_routine_load_json ON
test_routineload_tbl
COLUMNS(user_id,name,age)
PROPERTIES(
  "format"="json",
  "jsonpaths"=["$.user_id","$.name","$.age"]
)
FROM KAFKA(
  "kafka_broker_list" = "192.168.88.62:9092",
  "kafka_topic" = "test-routine-load-json",
  "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

:::info 备注 如果需要将 JSON 文件中根节点的 JSON 对象导入，jsonpaths 需要指定为 \$.，如：PROPERTIES("jsonpaths"="\$.")。 :::

查看导入状态

Routine Load 的状态分为两个维度：

- **导入作业：**用于查看导入任务目标表、子任务数量、导入延迟状态、导入配置与导入结果等信息。
- **导入任务：**用于查看导入的子任务状态、消费进度以及下发的 BE 节点。

查看导入作业

通过 SHOW ROUTINE LOAD 命令查看导入作业情况。该命令描述了当前作业的基本情况，如导入目标表、导入延迟状态、导入配置信息、导入错误信息等。

通过以下命令可以查看 testdb.example_routine_load_csv 的作业情况：

```
mysql> SHOW ROUTINE LOAD FOR testdb.example_routine_load\G
***** 1. row *****
```

Id: 12025

Name: example_routine_load

CreateTime: 2024-01-15 08:12:42

PauseTime: NULL

EndTime: NULL

DbName: default_cluster:testdb

TableName: test_routineload_tbl

IsMultiTable: false

State: RUNNING

DataSourceType: KAFKA

CurrentTaskNum: 1

JobProperties:

```
{"max_batch_rows":"200000","timezone":"America/New_York","send_batch_parallelism":
"1","load_to_single_tablet":"false","column_separator":"","","line_delimiter":"\
n","current_concurrent_number":"1","delete":"*","partial_columns":"false","merge_type
":"APPEND","exec_mem_limit":"2147483648","strict_mode":"false","jsonpaths":"","max_b
atch_interval":"10","max_batch_size":"104857600","fuzzy_parse":"false","partitions":"","
columnToColumnExpr":"user_id,name,age","whereExpr":"*","desired_concurrent_numbe
r":"5","precedingFilter":"","format":"csv","max_error_number":"0","max_filter_ratio":"1.
0","json_root":"","strip_outer_array":"false","num_as_string":"false"}
```

DataSourceProperties: {"topic":"test-
topic","currentKafkaPartitions":"0","brokerList":"192.168.88.62:9092"}

CustomProperties:

```
{"kafka_default_offsets":"OFFSET_BEGINNING","group.id":"example_routine_load_73daf6
00-884e-46c0-a02b-4e49fdf3b4dc"}
```

Statistic: {"receivedBytes":28,"runningTxns":

```
[],"errorRows":0,"committedTaskNum":3,"loadedRows":3,"loadRowsRate":0,"abortedTask
Num":0,"errorRowsAfterResumed":0,"totalRows":3,"unselectedRows":0,"receivedBytesRat
e":0,"taskExecuteTimeMs":30069}
```

Progress: {"0":"2"}

Lag: {"0":0}

ReasonOfStateChanged:

ErrorLogUrls:

OtherMsg:

User: root

Comment:

1 row in set (0.00 sec)

查看导入子任务

通过 SHOW ROUTINE LOAD TASK 命令查看导入子任务情况。可以通过 WHERE JobName = 或 FOR [...] 指定例行导入作业（``省略时使用当前数据库）。该命令描述了当前作业下的子任务信息，如子任务状态、下发的 BE id 等。

通过以下命令可以查看 testdb.example_routine_load_csv 的子任务情况：

```
mysql> SHOW ROUTINE LOAD TASK WHERE jobname = 'example_routine_load_csv';
```

TaskId	TxnId	TxnStatus	JobId	CreateTime	ExecuteStartTime	Timeout	Beld	DataSourceProperties
8cf47e6a68ed4da3-8f45b431db50e466	195	PREPARE	12177	2024-01-15 12:20:41	2024-01-15 12:21:01	20	10429	{"4":1231,"9":2603}
f2d4525c54074aa2-b6478cf8daaeb393	196	PREPARE	12177	2024-01-15 12:20:41	2024-01-15 12:21:01	20	12109	{"1":1225,"6":1216}
cb870f1553864250-975279875a25fab6	-1	NULL	12177	2024-01-15 12:20:52				
68771fd8a1824637-90a9dac2a7a0075e	-1	NULL	12177	2024-01-15 12:20:52				
77112dfea5e54b0a-a10eab3d5b19e565	197	PREPARE	12177	2024-01-15 12:21:02	2024-01-15 12:21:02	20	12098	{"0":3000,"5":2622}

也可以使用以下语法：

```
mysql> SHOW ROUTINE LOAD TASK FOR testdb.example_routine_load_csv;
```

暂停导入作业

可以通过 PAUSE ROUTINE LOAD 命令暂停导入作业。暂停后作业进入 PAUSED 状态，但导入作业并未终止，可以通过 RESUME ROUTINE LOAD 命令重启导入作业。

```
PAUSE ROUTINE LOAD FOR testdb.example_routine_load_csv;
```

恢复导入作业

可以通过 RESUME ROUTINE LOAD 命令恢复处于 PAUSED 状态的导入作业。

```
RESUME ROUTINE LOAD FOR testdb.example_routine_load_csv;
```

修改导入作业

可以通过 ALTER ROUTINE LOAD 命令修改已创建的导入作业。修改前需先使用 PAUSE ROUTINE LOAD 暂停作业，修改完成后使用 RESUME ROUTINE LOAD 恢复作业。

例如，修改期望并行度参数 desired_concurrent_number，并修改 Kafka Topic 信息：

```
ALTER ROUTINE LOAD FOR testdb.example_routine_load_csv
PROPERTIES(
  "desired_concurrent_number" = "3"
)
FROM KAFKA(
  "kafka_broker_list" = "192.168.88.60:9092",
  "kafka_topic" = "test-topic"
);
```

取消导入作业

可以通过 STOP ROUTINE LOAD 命令停止并删除 Routine Load 导入作业。**删除后的导入作业无法被恢复**，也无法通过 SHOW ROUTINE LOAD 命令查看。

```
STOP ROUTINE LOAD FOR testdb.example_routine_load_csv;
```

绑定 Compute Group

在**存算分离模式**下，Routine Load 的 Compute Group 选择逻辑按优先级如下：

1. 选择 use db@cluster 语句指定的 Compute Group；
2. 选择用户属性 default_compute_group 指定的 Compute Group；
3. 从当前用户有权限的 Compute Group 中选择一个。

在**存算一体模式**下，选择用户属性 resource_tags.location 中指定的 Compute Group。如果用户属性中未指定，则使用名为 default 的 Compute Group。

:::caution 注意 Routine Load 作业的 Compute Group **只能在创建时指定**，作业创建后无法修改其绑定的 Compute Group。 :::

参考手册

导入命令语法

创建一个 Routine Load 常驻导入作业的语法如下：

```
CREATE ROUTINE LOAD [.] [ON]
[merge_type]
[load_properties]
[job_properties]
FROM KAFKA [data_source_properties]
[COMMENT ""]
```

各模块说明如下：

- 模块：db_name；说明：指定创建导入任务的数据库。
- 模块：job_name；说明：指定创建的导入任务名称，同一个 database 下不能有名字相同的任务。
- 模块：tbl_name；说明：指定需要导入的表的名称，可选参数。如果不指定，则采用动态表的方式，此时需要 Kafka 中的数据包含表名的信息。
- 模块：merge_type；说明：数据合并类型。默认值为 APPEND。共有三种选项：- APPEND：追加导入；- MERGE：合并导入；- DELETE：导入的数据皆为需要删除的数据。
- 模块：load_properties；说明：导入描述模块，包括以下组成部分：- column_separator 子句- columns_mapping 子句- preceding_filter 子句- where_predicates 子句- partitions 子句- delete_on 子句- order_by 子句
- 模块：job_properties；说明：用于指定 Routine Load 的通用导入参数。
- 模块：data_source_properties；说明：用于描述 Kafka 数据源属性。
- 模块：comment；说明：用于描述导入作业的备注信息。

导入参数说明

FE 配置参数

- 参数名称：max_routine_load_task_concurrent_num；默认值：256；动态配置：是；FE Master 独有配置：是；参数描述：限制 Routine Load 的导入作业最大子并发数量。建议维持默认值。设置过大可能导致并发任务数过多，占用集群资源。
- 参数名称：max_routine_load_task_num_per_be；默认值：1024；动态配置：是；FE Master 独有配置：是；参数描述：每个 BE 限制的最大并发 Routine Load 任务数。max_routine_load_task_num_per_be 应该小于 routine_load_thread_pool_size。
- 参数名称：max_routine_load_job_num；默认值：100；动态配置：是；FE Master 独有配置：是；参数描述：限制最大 Routine Load 作业数，包括 NEED_SCHEDULED、RUNNING、PAUSE。

- 参数名称: max_tolerable_backend_down_num; 默认值: 0; 动态配置: 是; FE Master 独有配置: 是; 参数描述: 只要有一个 BE 宕机, Routine Load 就无法自动恢复。在满足某些条件时, Doris 可以将 PAUSED 的任务重新调度, 转换为 RUNNING 状态。该参数为 0 表示只有所有 BE 节点都处于 alive 状态时才允许重新调度。
- 参数名称: period_of_auto_resume_min; 默认值: 5 (分钟); 动态配置: 是; FE Master 独有配置: 是; 参数描述: 自动恢复 Routine Load 的周期。

BE 配置参数

- 参数名称: max_consumer_num_per_group; 默认值: 3; 动态配置: 是; 描述: 一个子任务最多生成几个 consumer 进行消费。

导入配置参数

在创建 Routine Load 作业时, 可以通过 CREATE ROUTINE LOAD 命令指定不同模块的导入配置参数。

tbl_name 子句

指定需要导入的表的名称, 可选参数。

如果不指定, 则采用**动态表**的方式, 此时需要 Kafka 中的数据包含表名的信息。目前仅支持从 Kafka 的 Value 中获取动态表名, 且需要符合以下格式: 以 JSON 为例:

table_name|{"col1": "val1", "col2": "val2"}}, 其中 tbl_name 为表名, 以 | 作为表名和表数据的分隔符。CSV 格式的数据也是类似的, 如: table_name|val1,val2,val3。

:::caution 注意 这里的 table_name 必须和 Doris 中的表名一致, 否则会导致导入失败。动态表不支持 column_mapping 配置。 :::

merge_type 子句

可以通过 merge_type 模块指定数据合并的类型, 有三种选项:

- **APPEND**: 追加导入方式;
- **MERGE**: 合并导入方式。仅适用于 Unique Key 模型, 需要配合 [DELETE ON] 模块以标注 Delete Flag 列;
- **DELETE**: 导入的数据皆为需要删除的数据。

load_properties 子句

可以通过 load_properties 模块描述导入数据的属性, 具体语法如下:

[**COLUMNS TERMINATED BY** ,]
[**COLUMNS** ([, , , ...]),]
[**WHERE** ,]
[**PARTITION**(, [, , ...]),]
[**DELETE ON** ,]
[**ORDER BY** [, , , ...]]

具体模块对应参数如下：

- 子模块：COLUMNS TERMINATED BY；参数：``；说明：用于指定列分隔符，默认为 \t。例如指定逗号为分隔符：COLUMNS TERMINATED BY ","。空值处理注意：- 空值（null）需要用 \n 表示，a,\n,b 表示中间列是一个空值（null）；- 空字符串（''）直接将数据置空，a,,b 表示中间列是一个空字符串（''）。
- 子模块：COLUMNS；参数：``；说明：用于指定对应的列名。例如指定导入列 (k1, k2, k3)：COLUMNS(k1, k2, k3)。下列情况可以缺省 COLUMNS 子句：- CSV 中的列与表中的列一一对应；- JSON 中的 key 列与表中的列名相同。
- 参数：``；说明：在导入过程中，可以通过列映射进行列的过滤和转换。例如目标列 k4 基于 k3 列使用公式 k3+1 计算得出：COLUMNS(k1, k2, k3, k4 = k3 + 1)。详细内容参考数据转换。
- 子模块：WHERE；参数：``；说明：根据条件过滤导入的数据源。例如只导入 age > 30 的数据：WHERE age > 30。
- 子模块：PARTITION；参数：``；说明：指定导入目标表中的哪些 partition。如果不指定，会自动导入对应的 partition 中。例如导入 p1 与 p2 分区：PARTITION(p1, p2)。
- 子模块：DELETE ON；参数：``；说明：在 MERGE 导入模式下，使用 delete_expr 标记哪些列需要被删除。例如在 MERGE 时删除 age > 30 的列：DELETE ON age > 30。
- 子模块：ORDER BY；参数：``；说明：仅针对 Unique Key 模型生效。用于指定导入数据中的 Sequence Column 列，以保证数据顺序。例如指定 create_time 为 Sequence Column：ORDER BY create_time。详细参考数据更新/Sequence 列。

job_properties 子句

在创建 Routine Load 导入作业时，可以指定 job_properties 子句以指定导入作业的属性。语法如下：

```
PROPERTIES ('' = ''[, '' = '' ...])
```

job_properties 子句具体参数选项如下：

- 参数: `desired_concurrent_number`; 说明: 默认值: 256 单个导入子任务 (load task) 期望的并发度。在导入过程中, 期望的子任务并发度可能不等于实际并发度。实际并发度通过以下公式计算: $\min(\text{topic_partition_num}, \text{desired_concurrent_number}, \text{max_routine_load_task_concurrent_num})$, 其中: - `topic\_partition\_num`: Kafka Topic 的 partition 数量; - `desired\_concurrent\_number`: 设置的参数大小; - `max\_routine\_load\_task\_concurrent\_num`: FE 中设置的 Routine Load 最大任务并行度参数。
- 参数: `max_batch_interval`; 说明: 每个子任务的最大运行时间, 单位是秒, 必须大于 0, 默认值为 60(s)。`max_batch_interval/max_batch_rows/max_batch_size` 共同形成子任务执行阈值。任一参数达到阈值, 导入子任务结束, 并生成新的导入子任务。
- 参数: `max_batch_rows`; 说明: 每个子任务最多读取的行数。必须大于等于 200000, 默认 20000000。`max_batch_interval/max_batch_rows/max_batch_size` 共同形成子任务执行阈值。任一参数达到阈值, 导入子任务结束, 并生成新的导入子任务。
- 参数: `max_batch_size`; 说明: 每个子任务最多读取的字节数。单位是字节, 范围是 100MB 到 1GB, 默认 1G。`max_batch_interval/max_batch_rows/max_batch_size` 共同形成子任务执行阈值。任一参数达到阈值, 导入子任务结束, 并生成新的导入子任务。
- 参数: `max_error_number`; 说明: 采样窗口内允许的最大错误行数, 必须大于等于 0, 默认 0 (即不允许有错误行)。采样窗口为 `max_batch_rows * 10`。如果采样窗口内错误行数大于 `max_error_number`, 则会导致例行作业被暂停, 需要人工介入检查数据质量问题, 可通过 `SHOW ROUTINE LOAD` 命令中的 `ErrorLogUrls` 检查数据的质量问题。被 `where` 条件过滤掉的行不算错误行。
- 参数: `strict_mode`; 说明: 是否开启严格模式, 默认关闭。严格模式表示对于导入过程中的列类型转换进行严格过滤。开启后, 非空原始数据的列类型变换如果结果为 NULL 则会被过滤。过滤策略: - 某衍生列 (由函数转换生成而来), `Strict Mode` 对其不产生影响; - 当列类型需要转换, 错误的数据类型将被过滤掉, 可在 `SHOW ROUTINE LOAD` 的 `ErrorLogUrls` 中查看; - 对于导入的某列类型包含范围限制的, 如果原始数据能正常通过类型转换但无法通过范围限制的, `strict mode` 对其不产生影响。例如类型是 `decimal(1,0)`, 原始数据为 10, 则属于可以通过类型转换但不在列声明的范围内。详细内容参考严格模式。
- 参数: `timezone`; 说明: 指定导入作业所使用的时区。默认为使用 Session 的 `timezone` 参数。该参数会影响所有导入涉及的与时区有关的函数结果。
- 参数: `format`; 说明: 指定导入数据格式, 默认是 CSV, 支持 JSON 格式。

- 参数：jsonpaths；说明：当导入数据格式为 JSON 时，可以通过 jsonpaths 指定抽取 JSON 数据中的字段。例如：`"jsonpaths" = "[\"$.userid\", \"$.username\", \"$.age\", \"$.city\"]"`
- 参数：json_root；说明：当导入数据格式为 JSON 时，可以通过 json_root 指定 JSON 数据的根节点。Doris 将通过 json_root 抽取根节点的元素进行解析。默认为空。例如：`"json_root" = "$.RECORDS"`
- 参数：strip_outer_array；说明：当导入数据格式为 JSON 时，strip_outer_array 为 true 表示 JSON 数据以数组的形式展现，数据中的每一个元素将被视为一行数据。默认值是 false。通常 Kafka 中的 JSON 数据可能以数组形式表示，即在最外层中包含中括号 []，此时可以指定 `"strip_outer_array" = "true"`。如以下数据会被解析成两行：`[{"user_id":1,"name":"Emily","age":25}, {"user_id":2,"name":"Benjamin","age":35}]`
- 参数：send_batch_parallelism；说明：用于设置发送批量数据的并行度。如果并行度的值超过 BE 配置中的 max_send_batch_parallelism_per_job，那么作为协调点的 BE 将使用 max_send_batch_parallelism_per_job 的值。
- 参数：load_to_single_tablet；说明：支持一个任务只导入数据到对应分区的一个 tablet，默认值为 false。该参数只允许在对带有 random 分桶的 olap 表导数的时候设置。
- 参数：partial_columns；说明：指定是否开启部分列更新功能。默认值为 false。该参数只允许在表模型为 Unique 且采用 Merge on Write 时设置。一流多表不支持此参数。具体参考文档部分列更新。
- 参数：unique_key_update_mode；说明：指定 Unique Key 表的更新模式。可选值：UPSERT（默认）：标准的整行插入或更新操作。UPDATE_FIXED_COLUMNS：部分列更新，所有行更新相同的列。等同于 partial_columns=true。UPDATE_FLEXIBLE_COLUMNS：灵活部分列更新，每行可以更新不同的列。需要 JSON 格式且表必须设置 enable_unique_key_skip_bitmap_column=true。不能与 jsonpaths、fuzzy_parse、COLUMNS 子句或 WHERE 子句一起使用。详情参考部分列更新。
- 参数：partial_update_new_key_behavior；说明：在 Unique Merge on Write 表上进行部分列更新时，对新插入行的处理方式。有两种类型 APPEND、ERROR。- APPEND：允许插入新行数据；- ERROR：插入新行时导入失败并报错。
- 参数：max_filter_ratio；说明：采样窗口内允许的最大过滤率。必须在大于等于 0 到小于等于 1 之间。默认值是 1.0，表示可以容忍任何错误行。采样窗口为 max_batch_rows * 10。如果采样窗口内错误行数/总行数大于 max_filter_ratio，则

会导致例行作业被暂停，需要人工介入检查数据质量问题。被 where 条件过滤掉的行不算错误行。

- 参数：enclose；说明：指定包围符。当 CSV 数据字段中含有行分隔符或列分隔符时，为防止意外截断，可指定单字节字符作为包围符起到保护作用。例如列分隔符为 ,，包围符为 '，数据为 a,'b,c'，则 b,c 会被解析为一个字段。
- 参数：escape；说明：指定转义符。用于转义在字段中出现的与包围符相同的字符。例如数据为 a,'b,'c'，包围符为 '，希望 b,'c 被作为一个字段解析，则需要指定单字节转义符（如 \），将数据修改为 a,'b,\c'。

data_source_properties 子句

在创建 Routine Load 导入作业时，可以指定 data_source_properties 子句以指定 Kafka 数据源的属性。语法如下：

```
FROM KAFKA (''' = '''[, ''' = ''' ...])
```

data_source_properties 子句具体参数选项如下：

- 参数：kafka_broker_list；说明：指定 Kafka 的 broker 连接信息。格式为 :。多个 broker 之间以逗号分隔。例如："kafka_broker_list" = ":9092,:9092"
- 参数：kafka_topic；说明：指定要订阅的 Kafka 的 topic。一个导入作业仅能消费一个 Kafka Topic。
- 参数：kafka_partitions；说明：指定需要订阅的 Kafka Partition。如果不指定，则默认消费所有分区。
- 参数：kafka_offsets；说明：待消费的 Kafka Partition 中起始消费点（offset）。如果指定时间，则会从大于等于该时间的最近一个 offset 处开始消费。offset 可以指定为大于等于 0 的具体 offset，或使用以下格式：
 - OFFSET_BEGINNING：从有数据的位置开始订阅；
 - OFFSET_END：从末尾开始订阅；
 - 时间格式，如："2021-05-22 11:00:00"。如果没有指定，则默认从 OFFSET_END 开始订阅 topic 下的所有 partition。可以指定多个起始消费点，使用逗号分隔，如："kafka_offsets" = "101,0,OFFSET_BEGINNING,OFFSET_END" 或 "kafka_offsets" = "2021-05-22 11:00:00,2021-05-22 11:00:00"。注意：时间格式不能和 OFFSET 格式混用。
- 参数：property；说明：指定自定义 kafka 参数，功能等同于 kafka shell 中 --property 参数。当参数的 Value 为一个文件时，需要在 Value 前加上关键词 FILE:。创建文件可以参考 CREATE FILE 命令文档。更多支持的自定义参数，可以参考 librdkafka 的官方 CONFIGURATION 文档中 client 端的配置项。例如："property.client.id" = "12345"、"property.group.id" = "group_id_0"、"property.ssl.ca.location" = "FILE:ca.pem"。

通过配置 data_source_properties 中的 kafka property 参数，可以配置安全访问选项。目前 Doris 支持多种 Kafka 安全协议，如 plaintext（默认）、SSL、PLAIN、Kerberos 等。

导入状态

通过 SHOW ROUTINE LOAD 命令可以查看导入作业的状态：

```
SHOW [ALL] ROUTINE LOAD [FOR jobName];
```

返回结果集示例：

```
mysql> SHOW ROUTINE LOAD FOR testdb.example_routine_load\G
```

```
***** 1. row *****
```

```
Id: 12025
```

```
Name: example_routine_load
```

```
CreateTime: 2024-01-15 08:12:42
```

```
PauseTime: NULL
```

```
EndTime: NULL
```

```
DbName: default_cluster:testdb
```

```
TableName: test_routineload_tbl
```

```
IsMultiTable: false
```

```
State: RUNNING
```

```
DataSourceType: KAFKA
```

```
CurrentTaskNum: 1
```

```
JobProperties:
```

```
{"max_batch_rows":"200000","timezone":"America/New_York","send_batch_parallelism":  
"1","load_to_single_tablet":"false","column_separator":",","line_delimiter":\  
"n","current_concurrent_number":"1","delete":"*","partial_columns":"false","merge_type":  
"APPEND","exec_mem_limit":"2147483648","strict_mode":"false","jsonpaths":"","max_b  
atch_interval":"10","max_batch_size":"104857600","fuzzy_parse":"false","partitions":"*","  
columnToColumnExpr":"user_id,name,age","whereExpr":"*","desired_concurrent_numbe  
r":"5","precedingFilter":"*","format":"csv","max_error_number":"0","max_filter_ratio":"1.  
0","json_root":"","strip_outer_array":"false","num_as_string":"false"}
```

```
DataSourceProperties: {"topic":"test-  
topic","currentKafkaPartitions":"0","brokerList":"192.168.88.62:9092"}
```

```
CustomProperties:
```

```
{"kafka_default_offsets":"OFFSET_BEGINNING","group.id":"example_routine_load_73daf6  
00-884e-46c0-a02b-4e49fdf3b4dc"}
```

```
Statistic: {"receivedBytes":28,"runningTxns":
```

```
[],"errorRows":0,"committedTaskNum":3,"loadedRows":3,"loadRowsRate":0,"abortedTask  
Num":0,"errorRowsAfterResumed":0,"totalRows":3,"unselectedRows":0,"receivedBytesRat
```

```
e":0,"taskExecuteTimeMs":30069}
  Progress: {"0":"2"}
  Lag: {"0":0}
ReasonOfStateChanged:
  ErrorLogUrls:
  OtherMsg:
  User: root
  Comment:
1 row in set (0.00 sec)
```

具体显示结果说明如下：

- 结果列：Id；列说明：作业 ID。由 Doris 自动生成。
- 结果列：Name；列说明：作业名称。
- 结果列：CreateTime；列说明：作业创建时间。
- 结果列：PauseTime；列说明：最近一次作业暂停时间。
- 结果列：EndTime；列说明：作业结束时间。
- 结果列：DbName；列说明：对应数据库名称。
- 结果列：TableName；列说明：对应表名称。多表的情况下由于是动态表，因此不显示具体表名，会显示 multi-table。
- 结果列：IsMultiTable；列说明：是否为多表（一流多表）。
- 结果列：State；列说明：作业运行状态，共有 5 种，详见下文作业状态机。
- 结果列：DataSourceType；列说明：数据源类型：KAFKA。
- 结果列：CurrentTaskNum；列说明：当前子任务数量。
- 结果列：JobProperties；列说明：作业配置详情。
- 结果列：DataSourceProperties；列说明：数据源配置详情。
- 结果列：CustomProperties；列说明：自定义配置。
- 结果列：Statistic；列说明：作业运行状态统计信息。
- 结果列：Progress；列说明：作业运行进度。对于 Kafka 数据源，显示每个分区当前已消费的 offset。如 {"0":"2"} 表示 Kafka 分区 0 的消费进度为 2。
- 结果列：Lag；列说明：作业延迟状态。对于 Kafka 数据源，显示每个分区的消费延迟。如 {"0":10} 表示 Kafka 分区 0 的消费延迟为 10。
- 结果列：ReasonOfStateChanged；列说明：作业状态变更的原因。
- 结果列：ErrorLogUrls；列说明：被过滤的质量不合格的数据的查看地址。
- 结果列：OtherMsg；列说明：其他错误信息。

作业状态机

Routine Load 作业共有 5 种状态，状态之间的转换关系如下：

- 状态：NEED_SCHEDULE；含义：作业等待调度；触发方式：CREATE ROUTINE LOAD、RESUME ROUTINE LOAD 后初始进入；下一步可能状态：RUNNING
- 状态：RUNNING；含义：作业运行中，正在持续消费 Kafka；触发方式：调度成功后进入；下一步可能状态：PAUSED / STOPPED
- 状态：PAUSED；含义：作业被暂停，未终止；触发方式：手动 PAUSE ROUTINE LOAD，或异常自动暂停；下一步可能状态：NEED_SCHEDULE（手动 RESUME 或自动恢复）/ CANCELLED
- 状态：STOPPED；含义：作业已停止，**无法重启**；触发方式：STOP ROUTINE LOAD；下一步可能状态：终态
- 状态：CANCELLED；含义：作业已取消；触发方式：库表删除等异常；下一步可能状态：终态

导入示例

下表按使用场景列出了本节涉及的所有典型示例，方便快速跳转：

- 类别：数据质量与过滤；场景：设置导入最大容错率；适用问题：数据质量不稳定，存在脏数据需要容忍
- 类别：数据质量与过滤；场景：设置导入过滤条件；适用问题：仅导入满足条件的数据
- 类别：数据质量与过滤；场景：严格模式导入；适用问题：严格过滤类型转换错误
- 类别：消费控制；场景：从指定消费点消费数据；适用问题：需要精确控制 Kafka offset
- 类别：消费控制；场景：指定 Consumer Group 的 group.id 与 client.id；适用问题：需要自定义 Kafka 消费者标识
- 类别：数据写入控制；场景：导入指定分区数据；适用问题：仅写入目标表的指定分区
- 类别：数据写入控制；场景：设置导入时区；适用问题：处理跨时区时间字段
- 类别：数据写入控制；场景：设置 merge_type；适用问题：Unique Key 表的删除或合并写入
- 类别：数据转换；场景：导入完成列映射与衍生列计算；适用问题：在导入时进行字段计算
- 类别：数据转换；场景：导入包含包围符的数据；适用问题：CSV 字段中含有分隔符

- 类别：复杂格式；场景：JSON 格式导入；适用问题：Kafka 数据为 JSON 格式
- 类别：复杂格式；场景：导入复杂类型；适用问题：处理 Array/Map/Bitmap/HLL 等类型
- 类别：安全与多表；场景：Kafka 安全认证；适用问题：SSL/Kerberos/PLAIN 等认证场景
- 类别：安全与多表；场景：一流多表导入；适用问题：一个 Topic 数据写入多个 Doris 表

设置导入最大容错率

1. 导入数据样例：

```
1,Benjamin,18
2,Emily,20
3,Alexander,dirty_data
```

2. 建表结构：

```
CREATE TABLE demo.routine_test01 (
  id   INT      NOT NULL COMMENT "User ID",
  name VARCHAR(30) NOT NULL COMMENT "Name",
  age  INT      COMMENT "Age"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令：

```
CREATE ROUTINE LOAD demo.kafka_job01 ON routine_test01
COLUMNS TERMINATED BY ",",
PROPERTIES
(
  "max_filter_ratio"="0.5",
  "max_error_number"="100",
  "strict_mode"="true"
)
FROM KAFKA
(
  "kafka_broker_list"="10.16.10.6:9092",
  "kafka_topic"="routineLoad01",
  "property.kafka_default_offsets"="OFFSET_BEGINNING"
);
```

4. 导入结果:

```
mysql> select * from routine_test01;
+-----+-----+-----+
| id | name   | age |
+-----+-----+-----+
| 1 | Benjamin | 18 |
| 2 | Emily   | 20 |
+-----+-----+-----+
2 rows in set (0.01 sec)
```

从指定消费点消费数据

1. 导入数据样例:

```
1,Benjamin,18
2,Emily,20
3,Alexander,22
4,Sophia,24
5,William,26
6,Charlotte,28
```

2. 建表结构:

```
CREATE TABLE demo.routine_test02 (
  id INT NOT NULL COMMENT "User ID",
  name VARCHAR(30) NOT NULL COMMENT "Name",
  age INT COMMENT "Age"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令:

```
CREATE ROUTINE LOAD demo.kafka_job02 ON routine_test02
  COLUMNS TERMINATED BY ","
  FROM KAFKA
  (
    "kafka_broker_list" = "10.16.10.6:9092",
    "kafka_topic" = "routineLoad02",
    "kafka_partitions" = "0",
    "kafka_offsets" = "3"
  );
```

4. 导入结果:

```
mysql> select * from routine_test02;
+-----+-----+-----+
| id | name    | age |
+-----+-----+-----+
| 4 | Sophia  | 24 |
| 5 | William | 26 |
| 6 | Charlotte | 28 |
+-----+-----+-----+
3 rows in set (0.01 sec)
```

指定 Consumer Group 的 group.id 与 client.id

1. 导入数据样例:

```
1,Benjamin,18
2,Emily,20
3,Alexander,22
```

2. 建表结构:

```
CREATE TABLE demo.routine_test03 (
  id INT NOT NULL COMMENT "User ID",
  name VARCHAR(30) NOT NULL COMMENT "Name",
  age INT COMMENT "Age"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令:

```
CREATE ROUTINE LOAD demo.kafka_job03 ON routine_test03
  COLUMNS TERMINATED BY ","
  FROM KAFKA
(
  "kafka_broker_list" = "10.16.10.6:9092",
  "kafka_topic" = "routineLoad01",
  "property.group.id" = "kafka_job03",
  "property.client.id" = "kafka_client_03",
  "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

4. 导入结果:

```
mysql> select * from routine_test03;
+-----+-----+-----+
| id | name   | age |
+-----+-----+-----+
| 1 | Benjamin | 18 |
| 2 | Emily   | 20 |
| 3 | Alexander | 22 |
+-----+-----+-----+
3 rows in set (0.01 sec)
```

设置导入过滤条件

1. 导入数据样例:

```
1,Benjamin,18
2,Emily,20
3,Alexander,22
4,Sophia,24
5,William,26
6,Charlotte,28
```

2. 建表结构:

```
CREATE TABLE demo.routine_test04 (
  id INT NOT NULL COMMENT "User ID",
  name VARCHAR(30) NOT NULL COMMENT "Name",
  age INT COMMENT "Age"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令:

```
CREATE ROUTINE LOAD demo.kafka_job04 ON routine_test04
  COLUMNS TERMINATED BY ",",
  WHERE id >= 3
  FROM KAFKA
(
  "kafka_broker_list" = "10.16.10.6:9092",
  "kafka_topic" = "routineLoad04",
```

```
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

4. 导入结果:

```
mysql> select * from routine_test04;
+-----+-----+
| id | name   | age |
+-----+-----+
| 4 | Sophia | 24 |
| 5 | William | 26 |
| 6 | Charlotte | 28 |
+-----+-----+
3 rows in set (0.01 sec)
```

导入指定分区数据

1. 导入数据样例:

```
1,Benjamin,18,2024-02-04 10:00:00
2,Emily,20,2024-02-05 11:00:00
3,Alexander,22,2024-02-06 12:00:00
```

2. 建表结构:

```
CREATE TABLE demo.routine_test05 (
  id INT NOT NULL COMMENT "ID",
  name VARCHAR(30) NOT NULL COMMENT "Name",
  age INT COMMENT "Age",
  date DATETIME COMMENT "Date"
)
DUPLICATE KEY(`id`)
PARTITION BY RANGE(`id`)
(PARTITION partition_a VALUES [("0"), ("1")),
PARTITION partition_b VALUES [("1"), ("2")),
PARTITION partition_c VALUES [("2"), ("3")])
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令:

```
CREATE ROUTINE LOAD demo.kafka_job05 ON routine_test05
  COLUMNS TERMINATED BY ",",
  PARTITION(partition_b)
```

```

FROM KAFKA
(
  "kafka_broker_list" = "10.16.10.6:9092",
  "kafka_topic" = "routineLoad05",
  "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);

```

4. 导入结果:

```

mysql> select * from routine_test05;
+-----+-----+-----+-----+
| id | name | age | date |
+-----+-----+-----+-----+
| 1 | Benjamin | 18 | 2024-02-04 10:00:00 |
+-----+-----+-----+-----+
1 rows in set (0.01 sec)

```

设置导入时区

1. 导入数据样例:

```

1,Benjamin,18,2024-02-04 10:00:00
2,Emily,20,2024-02-05 11:00:00
3,Alexander,22,2024-02-06 12:00:00

```

2. 建表结构:

```

CREATE TABLE demo.routine_test06 (
  id INT NOT NULL COMMENT "id",
  name VARCHAR(30) NOT NULL COMMENT "name",
  age INT COMMENT "age",
  date DATETIME COMMENT "date"
)
DUPLICATE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1;

```

3. 导入命令:

```

CREATE ROUTINE LOAD demo.kafka_job06 ON routine_test06
COLUMNS TERMINATED BY ","
PROPERTIES
(
  "timezone" = "Asia/Shanghai"
)

```

```

)
FROM KAFKA
(
  "kafka_broker_list" = "10.16.10.6:9092",
  "kafka_topic" = "routineLoad06",
  "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);

```

4. 导入结果：

```

mysql> select * from routine_test06;
+-----+-----+-----+-----+
| id | name   | age | date           |
+-----+-----+-----+-----+
| 1 | Benjamin | 18 | 2024-02-04 10:00:00 |
| 2 | Emily   | 20 | 2024-02-05 11:00:00 |
| 3 | Alexander | 22 | 2024-02-06 12:00:00 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

设置 merge_type

指定 merge_type 进行 delete 操作

1. 导入数据样例：

```

3,Alexander,22
5,William,26

```

导入前表中数据如下：

```

mysql> SELECT * FROM routine_test07;
+-----+-----+-----+
| id | name   | age |
+-----+-----+-----+
| 1 | Benjamin | 18 |
| 2 | Emily   | 20 |
| 3 | Alexander | 22 |
| 4 | Sophia  | 24 |
| 5 | William | 26 |
| 6 | Charlotte | 28 |
+-----+-----+-----+

```

2. 建表结构:

```
CREATE TABLE demo.routine_test07 (  
  id INT NOT NULL COMMENT "id",  
  name VARCHAR(30) NOT NULL COMMENT "name",  
  age INT COMMENT "age"  
)  
UNIQUE KEY(id)  
DISTRIBUTED BY HASH(id) BUCKETS 1;
```

3. 导入命令:

```
CREATE ROUTINE LOAD demo.kafka_job07 ON routine_test07  
  WITH DELETE  
  COLUMNS TERMINATED BY ","  
  FROM KAFKA  
  (  
    "kafka_broker_list" = "10.16.10.6:9092",  
    "kafka_topic" = "routineLoad07",  
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"  
  );
```

4. 导入结果:

```
mysql> SELECT * FROM routine_test07;
```

```
+-----+-----+-----+  
| id | name   | age |  
+-----+-----+-----+  
| 1 | Benjamin | 18 |  
| 2 | Emily   | 20 |  
| 4 | Sophia  | 24 |  
| 6 | Charlotte | 28 |  
+-----+-----+-----+
```

指定 merge_type 进行 merge 操作

1. 导入数据样例:

```
1,xiaoxiaoli,28  
2,xiaoxiaowang,30  
3,xiaoxiaoliu,32  
4,dadali,34
```

5,dadawang,36

6,dadaliu,38

导入前表中数据如下：

```
mysql> SELECT * FROM routine_test08;
```

```
+-----+-----+-----+
| id | name      | age |
+-----+-----+-----+
| 1 | Benjamin  | 18 |
| 2 | Emily     | 20 |
| 3 | Alexander | 22 |
| 4 | Sophia    | 24 |
| 5 | William   | 26 |
| 6 | Charlotte | 28 |
+-----+-----+-----+
6 rows in set (0.01 sec)
```

2. 建表结构：

```
CREATE TABLE demo.routine_test08 (
  id INT NOT NULL COMMENT "id",
  name VARCHAR(30) NOT NULL COMMENT "name",
  age INT COMMENT "age"
)
UNIQUE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1;
```

3. 导入命令：

```
CREATE ROUTINE LOAD demo.kafka_job08 ON routine_test08
  WITH MERGE
  COLUMNS TERMINATED BY ",",
  DELETE ON id = 2
  FROM KAFKA
  (
    "kafka_broker_list" = "10.16.10.6:9092",
    "kafka_topic" = "routineLoad08",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
  );
```

4. 导入结果：

```
mysql> SELECT * FROM routine_test08;
```

```
+-----+-----+-----+
| id | name      | age |
+-----+-----+-----+
| 1 | xiaoxiaoli | 28 |
| 3 | xiaoxiaoliu | 32 |
| 4 | dadali     | 34 |
| 5 | dadawang   | 36 |
| 6 | dadaliu    | 38 |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

指定导入需要 *merge* 的 *sequence* 列

1. 导入数据样例:

```
1,xiaoxiaoli,28
2,xiaoxiaowang,30
3,xiaoxiaoliu,32
4,dadali,34
5,dadawang,36
6,dadaliu,38
```

导入前表中数据如下:

```
mysql> SELECT * FROM routine_test09;
```

```
+-----+-----+-----+
| id | name      | age |
+-----+-----+-----+
| 1 | Benjamin  | 18 |
| 2 | Emily     | 20 |
| 3 | Alexander | 22 |
| 4 | Sophia    | 24 |
| 5 | William   | 26 |
| 6 | Charlotte | 28 |
+-----+-----+-----+
6 rows in set (0.01 sec)
```

2. 建表结构:

```
CREATE TABLE demo.routine_test08 (
  id INT NOT NULL COMMENT "id",
  name VARCHAR(30) NOT NULL COMMENT "name",
```

```

    age INT COMMENT "age"
)
UNIQUE KEY(id)
DISTRIBUTED BY HASH(id) BUCKETS 1
PROPERTIES (
    "function_column.sequence_col" = "age"
);

```

3. 导入命令:

```

CREATE ROUTINE LOAD demo.kafka_job09 ON routine_test09
WITH MERGE
COLUMNS TERMINATED BY ",",
COLUMNS(id, name, age),
DELETE ON id = 2,
ORDER BY age
PROPERTIES
(
    "desired_concurrent_number"="1",
    "strict_mode" = "false"
)
FROM KAFKA
(
    "kafka_broker_list" = "10.16.10.6:9092",
    "kafka_topic" = "routineLoad09",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);

```

4. 导入结果:

```
mysql> SELECT * FROM routine_test09;
```

```

+-----+-----+-----+
| id | name   | age |
+-----+-----+-----+
| 1 | xiaoxiaoli | 28 |
| 3 | xiaoxiaoliu | 32 |
| 4 | dadali   | 34 |
| 5 | dadawang | 36 |
| 6 | dadaliu  | 38 |
+-----+-----+-----+
5 rows in set (0.00 sec)

```

导入完成列映射与衍生列计算

1. 导入数据样例:

```
1,Benjamin,18
2,Emily,20
3,Alexander,22
```

2. 建表结构:

```
CREATE TABLE demo.routine_test10 (
  id INT NOT NULL COMMENT "id",
  name VARCHAR(30) NOT NULL COMMENT "name",
  age INT COMMENT "age",
  num INT COMMENT "number"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令:

```
CREATE ROUTINE LOAD demo.kafka_job10 ON routine_test10
  COLUMNS TERMINATED BY ",",
  COLUMNS(id, name, age, num=age*10)
  FROM KAFKA
  (
    "kafka_broker_list" = "10.16.10.6:9092",
    "kafka_topic" = "routineLoad10",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
  );
```

4. 导入结果:

```
mysql> SELECT * FROM routine_test10;
```

```
+-----+-----+-----+-----+
| id | name      | age | num |
+-----+-----+-----+-----+
| 1 | Benjamin  | 18 | 180 |
| 2 | Emily     | 20 | 200 |
| 3 | Alexander | 22 | 220 |
+-----+-----+-----+-----+
3 rows in set (0.01 sec)
```

导入包含包围符的数据

1. 导入数据样例:

```
1,"Benjamin",18
2,"Emily",20
3,"Alexander",22
```

2. 建表结构:

```
CREATE TABLE demo.routine_test11 (
  id INT NOT NULL COMMENT "id",
  name VARCHAR(30) NOT NULL COMMENT "name",
  age INT COMMENT "age",
  num INT COMMENT "number"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令:

```
CREATE ROUTINE LOAD demo.kafka_job11 ON routine_test11
  COLUMNS TERMINATED BY ","
  PROPERTIES
  (
    "desired_concurrent_number"="1",
    "enclose"="\\""
  )
  FROM KAFKA
  (
    "kafka_broker_list"="10.16.10.6:9092",
    "kafka_topic"="routineLoad12",
    "property.kafka_default_offsets"="OFFSET_BEGINNING"
  );
```

4. 导入结果:

```
mysql> SELECT * FROM routine_test11;
+-----+-----+-----+-----+
| id | name      | age | num |
+-----+-----+-----+-----+
| 1 | Benjamin  | 18 | 180 |
| 2 | Emily     | 20 | 200 |
```

```
| 3 | Alexander | 22 | 220 |
+-----+-----+-----+
3 rows in set (0.02 sec)
```

JSON 格式导入

以简单模式导入 JSON 格式数据

1. 导入数据样例:

```
{ "id": 1, "name": "Benjamin", "age": 18 }
{ "id": 2, "name": "Emily", "age": 20 }
{ "id": 3, "name": "Alexander", "age": 22 }
```

2. 建表结构:

```
CREATE TABLE demo.routine_test12 (
  id INT NOT NULL COMMENT "id",
  name VARCHAR(30) NOT NULL COMMENT "name",
  age INT COMMENT "age"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令:

```
CREATE ROUTINE LOAD demo.kafka_job12 ON routine_test12
  PROPERTIES
  (
    "format" = "json"
  )
  FROM KAFKA
  (
    "kafka_broker_list" = "10.16.10.6:9092",
    "kafka_topic" = "routineLoad12",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
  );
```

4. 导入结果:

```
mysql> select * from routine_test12;
+-----+-----+-----+
| id | name | age |
+-----+-----+-----+
```

```
| 1 | Benjamin | 18 |
| 2 | Emily    | 20 |
| 3 | Alexander | 22 |
+-----+-----+-----+
3 rows in set (0.02 sec)
```

匹配模式导入复杂的 JSON 格式数据

1. 导入数据样例：

```
{ "name" : "Benjamin", "id" : 1, "num":180 , "age":18 }
{ "name" : "Emily", "id" : 2, "num":200 , "age":20 }
{ "name" : "Alexander", "id" : 3, "num":220 , "age":22 }
```

2. 建表结构：

```
CREATE TABLE demo.routine_test13 (
  id INT NOT NULL COMMENT "id",
  name VARCHAR(30) NOT NULL COMMENT "name",
  age INT COMMENT "age",
  num INT COMMENT "num"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令：

```
CREATE ROUTINE LOAD demo.kafka_job13 ON routine_test13
  COLUMNS(name, id, num, age)
  PROPERTIES
  (
    "format" = "json",
    "jsonpaths" = "[\"$.name\", \"$.id\", \"$.num\", \"$.age\"]"
  )
  FROM KAFKA
  (
    "kafka_broker_list" = "10.16.10.6:9092",
    "kafka_topic" = "routineLoad13",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
  );
```

4. 导入结果：

```
mysql> select * from routine_test13;
```

```
+-----+-----+-----+-----+
| id | name      | age | num |
+-----+-----+-----+-----+
| 1 | Benjamin  | 18 | 180 |
| 2 | Emily     | 20 | 200 |
| 3 | Alexander | 22 | 220 |
+-----+-----+-----+-----+
3 rows in set (0.01 sec)
```

指定 JSON 根节点导入数据

1. 导入数据样例:

```
{"id": 1231, "source": {"id": 1, "name": "Benjamin", "age": 18 }}
{"id": 1232, "source": {"id": 2, "name": "Emily", "age": 20 }}
{"id": 1233, "source": {"id": 3, "name": "Alexander", "age": 22 }}
```

2. 建表结构:

```
CREATE TABLE demo.routine_test14 (
  id INT NOT NULL COMMENT "id",
  name VARCHAR(30) NOT NULL COMMENT "name",
  age INT COMMENT "age"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令:

```
CREATE ROUTINE LOAD demo.kafka_job14 ON routine_test14
  PROPERTIES
  (
    "format" = "json",
    "json_root" = "$.source"
  )
FROM KAFKA
(
  "kafka_broker_list" = "10.16.10.6:9092",
  "kafka_topic" = "routineLoad14",
  "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

4. 导入结果:

```
mysql> select * from routine_test14;
+-----+-----+-----+
| id | name      | age |
+-----+-----+-----+
| 1 | Benjamin  | 18 |
| 2 | Emily     | 20 |
| 3 | Alexander | 22 |
+-----+-----+-----+
3 rows in set (0.01 sec)
```

导入完成列映射与衍生列计算 (JSON)

1. 导入数据样例:

```
{ "id": 1, "name": "Benjamin", "age": 18 }
{ "id": 2, "name": "Emily", "age": 20 }
{ "id": 3, "name": "Alexander", "age": 22 }
```

2. 建表结构:

```
CREATE TABLE demo.routine_test15 (
  id INT NOT NULL COMMENT "id",
  name VARCHAR(30) NOT NULL COMMENT "name",
  age INT COMMENT "age",
  num INT COMMENT "num"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令:

```
CREATE ROUTINE LOAD demo.kafka_job15 ON routine_test15
  COLUMNS(id, name, age, num=age*10)
  PROPERTIES
  (
    "format" = "json"
  )
  FROM KAFKA
  (
    "kafka_broker_list" = "10.16.10.6:9092",
    "kafka_topic" = "routineLoad15",
```

```
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

4. 导入结果：

```
mysql> select * from routine_test15;
+-----+-----+-----+
| id | name      | age | num |
+-----+-----+-----+
| 1 | Benjamin  | 18 | 180 |
| 2 | Emily     | 20 | 200 |
| 3 | Alexander | 22 | 220 |
+-----+-----+-----+
3 rows in set (0.01 sec)
```

灵活部分列更新

本示例演示如何使用灵活部分列更新，其中每行可以更新不同的列。这在 **CDC 场景** 中非常有用，因为变更记录可能包含不同的字段。

1. 导入数据样例（每条 JSON 记录更新不同的列）：

```
{"id": 1, "balance": 150.00, "last_active": "2024-01-15 10:30:00"}
{"id": 2, "city": "Shanghai", "age": 28}
{"id": 3, "name": "Alice", "balance": 500.00, "city": "Beijing"}
{"id": 1, "age": 30}
{"id": 4, "__DORIS_DELETE_SIGN__": 1}
```

2. 建表（必须启用 Merge-on-Write 和 skip bitmap 列）：

```
CREATE TABLE demo.routine_test_flexible (
  id      INT      NOT NULL COMMENT "id",
  name    VARCHAR(30)  COMMENT "姓名",
  age     INT        COMMENT "年龄",
  city    VARCHAR(50)  COMMENT "城市",
  balance DECIMAL(10,2) COMMENT "余额",
  last_active DATETIME COMMENT "最后活跃时间"
)
UNIQUE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1
PROPERTIES (
  "replication_num" = "1",
```

```
"enable_unique_key_merge_on_write" = "true",  
"enable_unique_key_skip_bitmap_column" = "true"  
);
```

3. 插入初始数据:

```
INSERT INTO demo.routine_test_flexible VALUES  
(1, 'John', 25, 'Shenzhen', 100.00, '2024-01-01 08:00:00'),  
(2, 'Jane', 30, 'Guangzhou', 200.00, '2024-01-02 09:00:00'),  
(3, 'Bob', 35, 'Hangzhou', 300.00, '2024-01-03 10:00:00'),  
(4, 'Tom', 40, 'Nanjing', 400.00, '2024-01-04 11:00:00');
```

4. 导入命令:

```
CREATE ROUTINE LOAD demo.kafka_job_flexible ON routine_test_flexible  
  PROPERTIES  
  (  
    "format" = "json",  
    "unique_key_update_mode" = "UPDATE_FLEXIBLE_COLUMNS"  
  )  
FROM KAFKA  
  (  
    "kafka_broker_list" = "10.16.10.6:9092",  
    "kafka_topic" = "routineLoadFlexible",  
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"  
  );
```

5. 导入结果:

```
mysql> SELECT * FROM demo.routine_test_flexible ORDER BY id;
```

```
+-----+-----+-----+-----+-----+-----+  
| id | name | age | city | balance | last_active |  
+-----+-----+-----+-----+-----+-----+  
| 1 | John | 30 | Shenzhen | 150.00 | 2024-01-15 10:30:00 |  
| 2 | Jane | 28 | Shanghai | 200.00 | 2024-01-02 09:00:00 |  
| 3 | Alice | 35 | Beijing | 500.00 | 2024-01-03 10:00:00 |  
+-----+-----+-----+-----+-----+-----+
```

3 rows in set (0.01 sec)

:::info 注意 id=4 的行因为 __DORIS_DELETE_SIGN__ 被删除，每行只更新了其对应 JSON 记录中包含的列。 :::

导入复杂类型

导入 Array 数据类型

1. 导入数据样例：

```
{ "id": 1, "name": "Benjamin", "age": 18, "array": [1, 2, 3, 4, 5] }
{ "id": 2, "name": "Emily", "age": 20, "array": [6, 7, 8, 9, 10] }
{ "id": 3, "name": "Alexander", "age": 22, "array": [11, 12, 13, 14, 15] }
```

2. 建表结构：

```
CREATE TABLE demo.routine_test16
(
  id INT NOT NULL COMMENT "id",
  name VARCHAR(30) NOT NULL COMMENT "name",
  age INT COMMENT "age",
  array ARRAY NULL COMMENT "test array column"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令：

```
CREATE ROUTINE LOAD demo.kafka_job16 ON routine_test16
  PROPERTIES
  (
    "format" = "json"
  )
  FROM KAFKA
  (
    "kafka_broker_list" = "10.16.10.6:9092",
    "kafka_topic" = "routineLoad16",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
  );
```

4. 导入结果：

```
mysql> select * from routine_test16;
+-----+-----+-----+-----+
| id | name      | age | array      |
+-----+-----+-----+-----+
| 1 | Benjamin  | 18 | [1, 2, 3, 4, 5] |
```

```
| 2 | Emily      | 20 | [6, 7, 8, 9, 10] |
| 3 | Alexander   | 22 | [11, 12, 13, 14, 15] |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

导入 Map 数据类型

1. 导入数据样例：

```
{ "id": 1, "name": "Benjamin", "age": 18, "map": { "a": 100, "b": 200 } }
{ "id": 2, "name": "Emily", "age": 20, "map": { "c": 300, "d": 400 } }
{ "id": 3, "name": "Alexander", "age": 22, "map": { "e": 500, "f": 600 } }
```

2. 建表结构：

```
CREATE TABLE demo.routine_test17 (
  id INT NOT NULL COMMENT "id",
  name VARCHAR(30) NOT NULL COMMENT "name",
  age INT COMMENT "age",
  map Map NULL COMMENT "test column"
)
DUPLICATE KEY(`id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令：

```
CREATE ROUTINE LOAD demo.kafka_job17 ON routine_test17
PROPERTIES
(
  "format" = "json"
)
FROM KAFKA
(
  "kafka_broker_list" = "10.16.10.6:9092",
  "kafka_topic" = "routineLoad17",
  "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

4. 导入结果：

```
mysql> select * from routine_test17;
+-----+-----+-----+-----+
| id | name      | age | map      |
+-----+-----+-----+-----+
```

```
+-----+-----+-----+
| 1 | Benjamin | 18 | {"a":100, "b":200} |
| 2 | Emily    | 20 | {"c":300, "d":400} |
| 3 | Alexander | 22 | {"e":500, "f":600} |
+-----+-----+-----+
3 rows in set (0.01 sec)
```

导入 Bitmap 数据类型

1. 导入数据样例：

```
{ "id": 1, "name": "Benjamin", "age": 18, "bitmap_id": 243 }
{ "id": 2, "name": "Emily", "age": 20, "bitmap_id": 28574 }
{ "id": 3, "name": "Alexander", "age": 22, "bitmap_id": 8573 }
```

2. 建表结构：

```
CREATE TABLE demo.routine_test18 (
  id INT NOT NULL COMMENT "id",
  name VARCHAR(30) NOT NULL COMMENT "name",
  age INT COMMENT "age",
  bitmap_id INT COMMENT "test",
  device_id BITMAP BITMAP_UNION COMMENT "test column"
)
AGGREGATE KEY (`id`, `name`, `age`, `bitmap_id`)
DISTRIBUTED BY HASH(`id`) BUCKETS 1;
```

3. 导入命令：

```
CREATE ROUTINE LOAD demo.kafka_job18 ON routine_test18
  COLUMNS(id, name, age, bitmap_id, device_id=to_bitmap(bitmap_id))
  PROPERTIES
  (
    "format" = "json"
  )
  FROM KAFKA
  (
    "kafka_broker_list" = "10.16.10.6:9092",
    "kafka_topic" = "routineLoad18",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
  );
```

4. 导入结果：

```
mysql> select id, BITMAP_UNION_COUNT(pv) over(order by id) uv from(
-> select id, BITMAP_UNION(device_id) as pv
-> from routine_test18
-> group by id
-> ) final;
+-----+-----+
| id | uv |
+-----+-----+
| 1 | 1 |
| 2 | 2 |
| 3 | 3 |
+-----+-----+
3 rows in set (0.00 sec)
```

导入 HLL 数据类型

1. 导入数据样例:

```
2022-05-05,10001,Test01,Beijing,windows
2022-05-05,10002,Test01,Beijing,linux
2022-05-05,10003,Test01,Beijing,macos
2022-05-05,10004,Test01,Hebei,windows
2022-05-06,10001,Test01,Shanghai,windows
2022-05-06,10002,Test01,Shanghai,linux
2022-05-06,10003,Test01,Jiangsu,macos
2022-05-06,10004,Test01,Shaanxi,windows
```

2. 建表结构:

```
create table demo.routine_test19 (
  dt    DATE,
  id    INT,
  name  VARCHAR(10),
  province VARCHAR(10),
  os    VARCHAR(10),
  pv    hll hll_union
)
Aggregate KEY (dt,id,name,province,os)
distributed by hash(id) buckets 10;
```

3. 导入命令:

```
CREATE ROUTINE LOAD demo.kafka_job19 ON routine_test19
  COLUMNS TERMINATED BY ",",
  COLUMNS(dt, id, name, province, os, pv=hll_hash(id))
  FROM KAFKA
  (
    "kafka_broker_list" = "10.16.10.6:9092",
    "kafka_topic" = "routineLoad19",
    "property.kafka_default_offsets" = "OFFSET_BEGINNING"
  );
```

4. 导入结果：

```
mysql> select * from routine_test19;
+-----+-----+-----+-----+-----+-----+
| dt      | id   | name  | province | os    | pv    |
+-----+-----+-----+-----+-----+-----+
| 2022-05-05 | 10001 | Test01 | Beijing  | windows | NULL |
| 2022-05-06 | 10001 | Test01 | Shanghai | windows | NULL |
| 2022-05-05 | 10002 | Test01 | Beijing  | linux   | NULL |
| 2022-05-06 | 10002 | Test01 | Shanghai | linux   | NULL |
| 2022-05-05 | 10004 | Test01 | Heibei   | windows | NULL |
| 2022-05-06 | 10004 | Test01 | Shanxi   | windows | NULL |
| 2022-05-05 | 10003 | Test01 | Beijing  | macos   | NULL |
| 2022-05-06 | 10003 | Test01 | Jiangsu  | macos   | NULL |
+-----+-----+-----+-----+-----+-----+
8 rows in set (0.01 sec)
```

```
mysql> SELECT HLL_UNION_AGG(pv) FROM routine_test19;
+-----+
| hll_union_agg(pv) |
+-----+
| 4 |
+-----+
1 row in set (0.01 sec)
```

Kafka 安全认证

Doris 支持以下几种 Kafka 安全协议接入方式，下面分别给出导入示例与参数说明。

导入 SSL 认证的 Kafka 数据

导入命令样例：

```

CREATE ROUTINE LOAD demo.kafka_job20 ON routine_test20
  PROPERTIES
  (
    "format" = "json"
  )
FROM KAFKA
  (
    "kafka_broker_list" = "192.168.100.129:9092",
    "kafka_topic" = "routineLoad21",
    "property.security.protocol" = "ssl",
    "property.ssl.ca.location" = "FILE:ca.pem",
    "property.ssl.certificate.location" = "FILE:client.pem",
    "property.ssl.key.location" = "FILE:client.key",
    "property.ssl.key.password" = "ssl_passwd"
  );

```

参数说明：

- 参数：property.security.protocol；介绍：使用的安全协议，如上述例子使用的是 SSL。
- 参数：property.ssl.ca.location；介绍：CA（Certificate Authority）证书的位置。
- 参数：property.ssl.certificate.location；介绍：（如果 Kafka server 端开启了 client 认证才需要配置）Client 的 public key 的位置。
- 参数：property.ssl.key.location；介绍：（如果 Kafka server 端开启了 client 认证才需要配置）Client 的 private key 的位置。
- 参数：property.ssl.key.password；介绍：（如果 Kafka server 端开启了 client 认证才需要配置）Client 的 private key 的密码。

导入 Kerberos 认证的 Kafka 数据

导入命令样例：

```

CREATE ROUTINE LOAD demo.kafka_job21 ON routine_test21
  PROPERTIES
  (
    "format" = "json"
  )
FROM KAFKA
  (
    "kafka_broker_list" = "192.168.100.129:9092",

```

```
"kafka_topic" = "routineLoad21",  
"property.security.protocol" = "SASL_PLAINTEXT",  
"property.sasl.kerberos.service.name" = "kafka",  
"property.sasl.kerberos.keytab" = "/opt/third/kafka/kerberos/kafka_client.keytab",  
"property.sasl.kerberos.principal" = "clients/stream.dt.local@EXAMPLE.COM"  
);
```

参数说明：

- 参数：property.security.protocol；介绍：使用的安全协议，如上述例子使用的是 SASL_PLAINTEXT。
- 参数：property.sasl.kerberos.service.name；介绍：指定 broker service name，默认是 Kafka。
- 参数：property.sasl.kerberos.keytab；介绍：keytab 文件的位置。
- 参数：property.sasl.kerberos.principal；介绍：指定 kerberos principal。

:::tip 提示 建议在 krb5.conf 中设置 rdns=true。否则可能会出现报错：Server kafka/15.5.4.68@EXAMPLE.COM not found in Kerberos database :::

导入 PLAIN 认证的 Kafka 集群

导入命令样例：

```
CREATE ROUTINE LOAD demo.kafka_job22 ON routine_test22  
  PROPERTIES  
  (  
    "format" = "json"  
  )  
FROM KAFKA  
  (  
    "kafka_broker_list" = "192.168.100.129:9092",  
    "kafka_topic" = "routineLoad22",  
    "property.security.protocol" = "SASL_PLAINTEXT",  
    "property.sasl.mechanism" = "PLAIN",  
    "property.sasl.username" = "admin",  
    "property.sasl.password" = "admin"  
  )  
);
```

参数说明：

- 参数：property.security.protocol；介绍：使用的安全协议，如上述例子使用的是 SASL_PLAINTEXT。

- 参数：property.sasl.mechanism；介绍：指定 SASL 认证机制为 PLAIN。
- 参数：property.sasl.username；介绍：SASL 的用户名。
- 参数：property.sasl.password；介绍：SASL 的密码。

连接加密认证的 Kafka 服务 (StreamNative 示例)

以访问 StreamNative 消息服务为例：

```
CREATE ROUTINE LOAD example_db.test1 ON example_tbl
COLUMNS(user_id, name, age)
FROM KAFKA (
  "kafka_broker_list" = "pc-xxxx.aws-mec1-test-xwiqv.aws.snio.cloud:9093",
  "kafka_topic" = "my_topic",
  "property.security.protocol" = "SASL_SSL",
  "property.sasl.mechanism" = "PLAIN",
  "property.sasl.username" = "user",
  "property.sasl.password" = "token:eyJhbxxx",
  "property.group.id" = "my_group_id_1",
  "property.client.id" = "my_client_id_1",
  "property.enable.ssl.certificate.verification" = "false"
);
```

:::caution 注意

- 如果没有在 BE 端配置信任的 CA 证书路径，需设置
"property.enable.ssl.certificate.verification" = "false"，不验证服务器证书是否可信；
- 否则，需配置信任的 CA 证书路径："property.ssl.ca.location" = "/path/to/ca-cert.pem"。 :::

一流多表导入

为 example_db 创建一个名为 test1 的 Kafka 例行动态多表导入任务。指定 group.id 和 client.id，并且自动默认消费所有分区，且从有数据的位置 (OFFSET_BEGINNING) 开始订阅。

假设需要将 Kafka 中的数据导入到 example_db 中的 tbl1 以及 tbl2 表中，可以创建一个名为 test1 的例行导入任务，将名为 my_topic 的 Kafka Topic 数据同时导入到 tbl1 和 tbl2 中：

```
CREATE ROUTINE LOAD example_db.test1
FROM KAFKA
```

```
(
  "kafka_broker_list" = "broker1:9092,broker2:9092,broker3:9092",
  "kafka_topic" = "my_topic",
  "property.kafka_default_offsets" = "OFFSET_BEGINNING"
);
```

此时需要 Kafka 中的数据包含表名信息。目前仅支持从 Kafka 的 Value 中获取动态表名，且需要符合以下格式：以 JSON 为例：table_name|{"col1": "val1", "col2": "val2"}，其中 tbl_name 为表名，以 | 作为表名和表数据的分隔符。CSV 格式的数据也是类似的，如：table_name|val1,val2,val3。

:::caution 注意 这里的 table_name 必须和 Doris 中的表名一致，否则会导致导入失败。动态表不支持 column_mapping 配置。 :::

严格模式导入

为 example_db 的 example_tbl 创建一个名为 test1 的 Kafka 例行导入任务，开启严格模式：

```
CREATE ROUTINE LOAD example_db.test1 ON example_tbl
COLUMNS(k1, k2, k3, v1, v2, v3 = k1 * 100),
PRECEDING FILTER k1 = 1,
WHERE k1 ` 查看 `ReasonOfStateChanged` 与 `ErrorLogUrls` ;
2. 如为脏数据，可适当调大 `max_filter_ratio` 与 `max_error_number` ;
3. 如为 Kafka 异常，确认 broker 与 topic 可达后执行 `RESUME ROUTINE LOAD` 恢复。
```

报错 `Offset out of range` 或 `out of range` 怎么处理？

通常是已记录的 offset 在 Kafka 中已被清理（被保留期淘汰）：

1. 暂停作业 `PAUSE ROUTINE LOAD FOR` ；
2. 通过 `ALTER ROUTINE LOAD` 将 `kafka\_offsets` 重置为有效位置（如 `OFFSET\_BEGINNING` 或具体 offset）；
3. 执行 `RESUME ROUTINE LOAD` 恢复作业。

报错 `Server kafka/xxx@EXAMPLE.COM not found in Kerberos database` ？

在 Kerberos 认证场景下，建议在 `krb5.conf` 中设置 `rdnbs=true` 。

出现 `Server certificate verification failed` 等 SSL 错误？

未在 BE 端配置可信的 CA 证书时，可设置 `"property.enable.ssl.certificate.validation" = "false"`，或显式指定 `"property.ssl.ca.location" = "/path/to/ca-cert.pem"`。

期望的并发度（desired_concurrent_number）没有生效？

实际并发度由以下三者中的最小值决定：

```
```text
min(topic_partition_num, desired_concurrent_number,
max_routine_load_task_concurrent_num)
```

请检查 Kafka Topic 的 partition 数和 FE 配置 max\_routine\_load\_task\_concurrent\_num。

**旧版本 Kafka**（形式提供表名，且表名需与 Doris 中表名严格一致。动态表不支持 `column_mapping`）。

**如何查看导入失败的脏数据？**

通过 SHOW ROUTINE LOAD FOR 命令查看返回结果中的 ErrorLogUrls 字段，浏览器或 wget 访问该 URL 即可获取被过滤的错误数据样本及报错原因。

**修改 Routine Load 作业必须先暂停吗？**

是的。ALTER ROUTINE LOAD 命令要求作业处于 PAUSED 状态。修改流程：PAUSE ROUTINE LOAD → ALTER ROUTINE LOAD → RESUME ROUTINE LOAD。

**更多帮助**

- SQL 手册参考：CREATE ROUTINE LOAD
- 在客户端命令行下输入 HELP ROUTINE LOAD 获取更多帮助信息。